



MÓDULO 3 · T08

Datos y API: Fetch y async/await

 *Consumir la PokéAPI*

Carlos Rodríguez Contreras · GitHub: `karrocon`

¿Qué aprenderás hoy?

En esta sesión aprenderás a consumir APIs REST desde TypeScript usando fetch con async/await. Al terminar, serás capaz de tipar respuestas, encadenar llamadas y manejar errores de forma robusta en proyectos reales.

1

fetch con async/await

Usar fetch con async/await en TypeScript para realizar peticiones HTTP de forma legible.

2

Tipar respuestas

Definir interfaces y tipos estrictos para las respuestas de la API y evitar errores en tiempo de compilación.

3

Llamadas encadenadas

Encadenar llamadas: lista de Pokémon → detalles por cada entrada usando `Promise.all`.

4

Manejo de errores

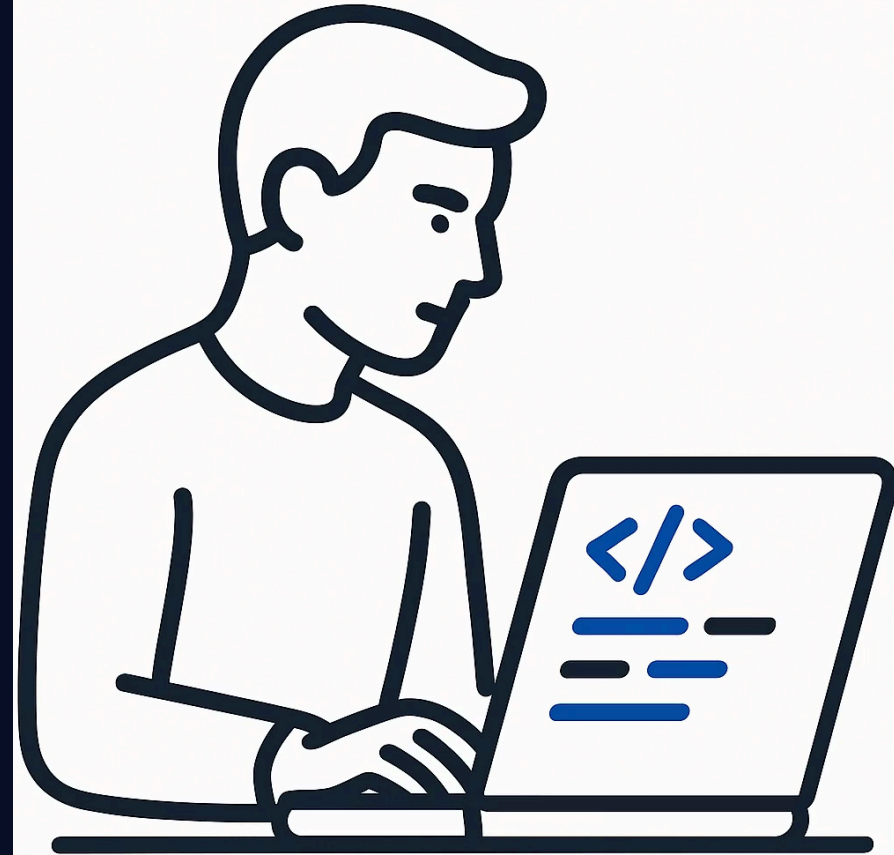
Usar `try/catch` y verificar `response.ok` para gestionar errores de red y HTTP.

fetch básico

La función `fetch` devuelve una promesa que resuelve con un objeto `Response`. Con `async/await` la lectura es secuencial y limpia. Es importante entender qué errores lanza `fetch` y cuáles no.

```
async function fetchPokemon(id: number) {  
  const response = await fetch(  
    `https://pokeapi.co/api/v2/pokemon/${id}`  
  );  
  const data = await response.json();  
  return data;  
}
```

❗ `fetch` solo lanza un error si hay un problema de red. Un 404 o 500 **no** lanza excepción – hay que verificar `response.ok` manualmente.



Verificar response.ok

¿Por qué verificar response.ok?

Un status 404 o 500 no hace que fetch lance un error. Si no compruebas `response.ok`, intentarás parsear una respuesta de error como si fuera datos válidos.

Verificar `response.ok` y lanzar manualmente garantiza que tu bloque `catch` reciba todos los fallos, tanto de red como HTTP.

- ✓ El patrón `if (!response.ok) throw new Error(...)` centraliza el manejo de errores en un solo lugar.

Patrón recomendado

```
async function fetchPokemon(id: number) {  
  const response = await fetch(  
    `https://pokeapi.co/api/v2/pokemon/${id}`  
  );  
  if (!response.ok) {  
    throw new Error(  
      `HTTP ${response.status} para pokemon ${id}`  
    );  
  }  
  const data = await response.json();  
  return data;  
}
```

Tipos de respuesta de la API

Definir tipos para las respuestas de la API permite que TypeScript verifique el acceso a propiedades en tiempo de compilación. Si la API cambia, el compilador te avisa de inmediato en todos los puntos donde se usan esos tipos.

```
type PokemonListResponse = {  
  count: number;  
  next: string | null;  
  results: { name: string; url: string }[];  
};
```

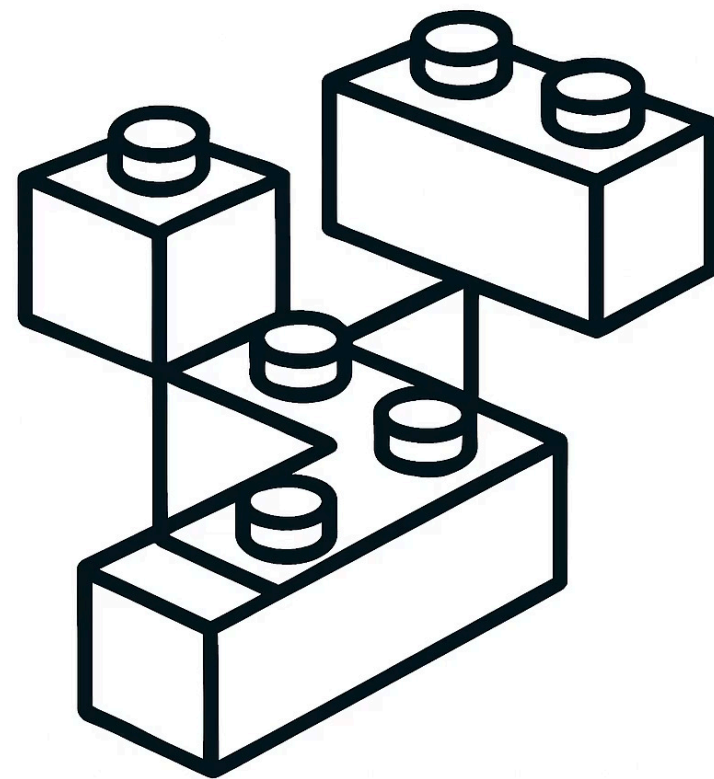
```
type PokemonDetailsResponse = {  
  id: number;  
  name: string;  
  types: { type: { name: string } }[];  
  sprites: { front_default: string };  
};
```

Promise.all — paralelo vs. secuencial

Cuando necesitas obtener detalles de varios Pokémon, hacerlo uno a uno es lento porque cada petición espera a que termine la anterior. `Promise.all` lanza todas las peticiones a la vez y espera a que todas resuelvan – el tiempo total es el de la petición más lenta, no la suma de todas.

```
// Secuencial (lento) — uno a uno
for (const p of list.results) {
  const detail = await fetch(p.url).then(r => r.json());
}
```

```
// Paralelo (rápido) — todos a la vez
const details = await Promise.all(
  list.results.map(async (p) => {
    const res = await fetch(p.url);
    return res.json();
  })
);
```



Función completa: fetchPokemons

Esta es la implementación completa tal y como aparece en la aplicación de ejemplo. Observa cómo se combinan la verificación de `response.ok`, el tipado explícito de la respuesta y `Promise.all` para obtener todos los detalles en paralelo antes de mapear al tipo local.

```
export async function fetchPokemons(
  limit = 20,
  offset = 0
): Promise<Pokemon[]> {
  try {
    const listRes = await fetch(
      `https://pokeapi.co/api/v2/pokemon?limit=${limit}&offset=${offset}`
    );
    if (!listRes.ok) throw new Error(`HTTP ${listRes.status}`);
    const list: PokemonListResponse = await listRes.json();

    const details = await Promise.all(
      list.results.map(async (entry) => {
        const detailRes = await fetch(entry.url);
        return detailRes.json() as Promise<PokemonDetailsResponse>;
      })
    );

    return details.map(mapToPokemon);
  } catch (error) {
    throw new Error(`Error al obtener pokémons: ${error}`);
  }
}
```

📌 **Buena práctica:** envuelve toda la función en un try/catch y relanza con un mensaje descriptivo para facilitar el diagnóstico desde el componente que la llame.

Función completa con tipado explícito

Tipado de la lista

```
const list: PokemonListResponse =  
  await listRes.json();  
  
// TypeScript conoce la forma exacta:  
// list.results → { name, url }[]  
// list.count → number  
// list.next → string | null
```

Anotar el tipo en `await listRes.json()` evita que TypeScript infiera `any` y habilita el autocompletado completo.

Cast en el detalle

```
return detailRes.json()  
  as Promise<PokemonDetailsResponse>;  
  
// Acceso seguro al resultado:  
// detail.id → number  
// detail.name → string  
// detail.types → { type: { name } }[]  
// detail.sprites → { front_default }
```

El cast con `as` le indica a TypeScript la forma que tendrá el JSON parseado para que los accesos posteriores sean seguros.

- ❗ Combinar el tipado de la lista y el cast del detalle garantiza que `mapToPokemon` reciba datos con la forma esperada – sin `any` en ningún punto de la cadena.

Tour starter/fetchPokemons.ts

Este es el fichero que encontrarás en la carpeta `starter/`. Contiene la firma correcta de la función con tres pasos marcados como comentarios `TODO` que deberás implementar durante el ejercicio. El cuerpo solo lanza un error como marcador de posición.

```
export async function fetchPokemons(  
  limit = 20,  
  offset = 0  
): Promise<Pokemon[]> {  
  // 1. fetch a la lista  
  // 2. por cada resultado, fetch a la URL de detalle  
  // 3. construye y devuelve el array  
  throw new Error('TODO: implementar fetchPokemons');  
}
```

⚠ No modifiques la firma de la función ni los tipos importados – céntrate en implementar los tres pasos marcados arriba.

✓ SOLUCIÓN

Solución — solucion/fetchPokemons.ts

La solución completa implementa los tres pasos: fetch a la lista con verificación de `response.ok`, `Promise.all` para los detalles en paralelo y el mapeo final al tipo local. Compara tu implementación con esta referencia.

```
export async function fetchPokemons(
  limit = 20,
  offset = 0
): Promise<Pokemon[]> {
  const listRes = await fetch(
    `https://pokeapi.co/api/v2/pokemon?limit=${limit}&offset=${offset}`
  );
  if (!listRes.ok) throw new Error(`HTTP ${listRes.status}`);
  const list = await listRes.json();

  const details = await Promise.all(
    list.results.map(async (entry: any) => {
      const dr = await fetch(entry.url);
      return dr.json();
    })
  );

  return details.map(mapToPokemon);
}
```

✓ Si el componente muestra la lista de Pokémon sin errores en consola y TypeScript no reporta ningún problema, ¡lo has conseguido! 🎉

Ejercicio T08

Pon en práctica todo lo aprendido. Abre el proyecto en tu editor y sigue los pasos en orden – cada uno construye sobre el anterior. El bonus es opcional pero muy recomendable para afianzar el patrón de carga progresiva.

1 Abre el fichero de inicio

Navega a `starter/fetchPokemons.ts` en tu editor.

2 Implementa la llamada a la lista

Haz fetch a `pokeapi.co/api/v2/pokemon` con `limit` y `offset` y verifica `response.ok`.

3 Usa Promise.all para los detalles

Mapea `list.results` a un array de promesas y resuélvelas todas en paralelo con `Promise.all`.

4 Mapea al tipo local

Aplica `mapToPokemon` sobre el array de detalles y devuelve el resultado tipado como `Pokemon[]`.

5 Bonus: botón "Cargar"

Llama a `fetchPokemons` desde un componente con un botón "Cargar" y muestra el listado en pantalla.

Resumen de la sesión

Estos son los cinco conceptos clave que has trabajado hoy. Dominarlos te permitirá consumir cualquier API REST de forma robusta, tipada y eficiente desde TypeScript.

Concepto	Sintaxis clave	Para qué sirve
fetch(url)	<code>await fetch(url)</code>	Petición HTTP asíncrona
response.ok	<code>if (!res.ok) throw ...</code>	Verificar status 200-299
response.json()	<code>await res.json()</code>	Parsear body como JSON
async/await	<code>async function f() { await ... }</code>	Sintaxis limpia para promesas
Promise.all([...])	<code>await Promise.all(arr.map(...))</code>	Llamadas en paralelo
try/catch	<code>try { ... } catch (e) { ... }</code>	Manejo de errores de red

✅ ¡Enhorabuena! Ahora tienes las herramientas para consumir APIs REST de forma tipada, eficiente y segura en cualquier proyecto TypeScript. 🚀